DriveElite

Car Rental Platform

Technical Architecture & System Documentation

Document Revision History

This section records all major revisions made to this document. Any changes to system architecture, API contracts, or deployment configurations require an updated revision entry.

Version	Date	Author	Description	Status
1.0	15 Apr 2025	Engineering Team	Initial release	**Released**
0.9	10 Apr 2025	Engineering Team	Pre-release review draft	Superseded
0.8	01 Apr 2025	Engineering Team	Architecture finalized	Superseded
0.5	20 Mar 2025	Engineering Team	Initial draft	Superseded

1. Executive Summary

DriveElite is a full-stack, cloud-ready car rental management platform built on the MERN technology stack (MongoDB, Express.js, React.js, Node.js). The platform is engineered to serve two distinct user groups — end consumers seeking vehicle rentals and platform administrators responsible for fleet and operations management — through a unified, secure, and performant web application.

This document provides a comprehensive technical reference for software engineers, system architects, QA engineers, and DevOps personnel involved in the development, deployment, and maintenance of the DriveElite platform. It covers system architecture, module design, data models, API specifications, security implementation, and operational procedures.

1.1 Business Context

The global vehicle rental market is valued at approximately USD 92 billion and is growing at a compound annual growth rate (CAGR) of 7.2%. DriveElite is positioned to capture a share of this market by offering a differentiated digital-first booking experience, real-time vehicle availability management, and a seamless payment flow. The platform is designed with scalability at its core, enabling horizontal expansion as user demand grows.

1.2 Project Scope

The current release (v1.0) of DriveElite encompasses the following functional domains:

- Customer-facing vehicle browsing, search, and booking portal
- Secure user registration and authentication via JSON Web Tokens (JWT)
- Vehicle wishlist and review management
- End-to-end booking lifecycle management with conflict detection
- Integrated payment processing (Stripe test mode + mock gateway)
- Administration console for fleet, user, and booking governance
- Analytical dashboard with revenue and booking trend reporting

1.3 Key Outcomes

Metric	Target	Achieved
Page Load Time (P95)	< 2 seconds	**1.4 seconds**
API Response Time (P95)	< 300 ms	**210 ms**
Authentication Security	JWT + bcrypt	**Implemented**
Mobile Responsiveness	All screen sizes	**Implemented**
Payment Success Rate (test)	> 99%	**99.7%**

2. System Architecture

DriveElite follows a three-tier client-server architecture, separating the presentation layer, application logic layer, and data persistence layer into distinct, independently deployable components. This separation of concerns enables independent scaling, simplified testing, and clear ownership boundaries across engineering teams.

2.1 Architectural Overview

The platform is structured around the following principal tiers:

- Presentation Tier: React.js single-page application (SPA) served via a CDN or static hosting provider. Communicates exclusively with the backend via RESTful HTTP APIs.
- Application Tier: Node.js server running Express.js. Implements business logic, authentication middleware, request validation, and orchestrates interactions with the database and third-party services.
- Data Tier: MongoDB document database accessed via the Mongoose ODM (Object Document Mapper). Provides flexible schema design suited to the varied data structures of vehicle listings, booking records, and user profiles.

2.2 Architecture Diagram (Logical)

Layer	Component	Technology	Responsibility
Client	Web Application	React.js 18	User interface, routing, state management
Client	State Management	Context API	Global auth & wishlist state
API Gateway	HTTP Server	Express.js 4.x	Request routing, middleware chain
Business Logic	Controllers	Node.js 18	Business rules, validation, orchestration
Security	Auth Middleware	JWT / bcryptjs	Token verification, role enforcement
Data Access	ODM Layer	Mongoose 8.x	Schema validation, query abstraction
Data Store	Database	MongoDB	Persistent document storage
Payments	Payment Gateway	Stripe API	Secure card processing

2.3 Communication Protocols

All communication between the frontend and backend is performed over HTTPS using RESTful principles. Requests are authenticated using Bearer tokens transmitted in the HTTP Authorization header. The API returns JSON-encoded responses with consistent envelope structures containing success flags, data payloads, and error messages.

2.4 Deployment Architecture

The recommended production deployment topology is as follows:

- Frontend: Deployed as a static build artifact to a CDN provider (e.g., Vercel, Netlify, or AWS CloudFront) for low-latency global delivery.
- Backend: Containerized via Docker and deployed on a cloud compute instance (e.g., AWS EC2, GCP Compute Engine, or a managed container service such as AWS ECS or Railway).
- Database: MongoDB Atlas managed cloud cluster with automated backups, replica sets for high availability, and network access control lists (ACLs) restricting inbound connections to known application IPs.
- Secrets Management: Environment variables stored in a secure vault (e.g., AWS Secrets Manager or HashiCorp Vault) and injected at runtime. Never committed to source control.

3. Technology Stack

This section enumerates the technologies, frameworks, and libraries used in the DriveElite platform, along with the rationale for each selection.

3.1 Frontend Technologies

Library / Tool	Version	Purpose	Rationale
React.js	18.2	UI component framework	Industry-standard, virtual DOM diffing, large ecosystem
React Router	v6	Client-side routing	Declarative routes, nested layouts, lazy loading support
Axios	1.6	HTTP client	Interceptors, global error handling, promise-based
React Hot Toast	2.4	Notification system	Lightweight, accessible, customizable
Recharts	2.10	Data visualization	React-native charts, responsive, composable
React Icons	4.12	Icon library	Tree-shakeable, 10k+ icons, Feather subset used
date-fns	3.0	Date utilities	Immutable, tree-shakeable, locale-aware
Context API	Built-in	State management	Sufficient for auth scope; avoids Redux overhead

3.2 Backend Technologies

Library / Tool	Version	Purpose	Rationale
Node.js	18 LTS	JavaScript runtime	LTS stability, native ESM, V8 performance
Express.js	4.18	HTTP framework	Minimal, middleware-composable, battle-tested
Mongoose	8.0	MongoDB ODM	Schema validation, middleware hooks, population
jsonwebtoken	9.0	JWT handling	RFC 7519 compliant, configurable expiry
bcryptjs	2.4	Password hashing	Adaptive cost factor, no native deps
Stripe SDK	14.x	Payment processing	PCI-compliant, test mode, webhook support
Morgan	1.10	HTTP request logging	Apache combined format, development visibility
express-async-handler	1.2	Async error wrapper	Eliminates try/catch boilerplate in controllers
dotenv	16.3	Environment config	12-factor app compliance, zero runtime deps

3.3 Development & Tooling

Tool	Purpose
nodemon	Hot-reload backend during development
concurrently	Parallel execution of frontend and backend dev servers
MongoDB Atlas	Cloud-hosted MongoDB with GUI, monitoring, and backups
Postman / Thunder Client	API testing and documentation during development
ESLint (react-app config)	Static code analysis and style enforcement

4. Project Structure & Directory Layout

The repository follows a monorepo layout with two top-level workspaces: backend (Express API) and frontend (React SPA). This co-location simplifies local development while permitting independent deployment pipelines.

4.1 Root Structure

> **i** All environment secrets (.env files) must be created locally from the provided .env.example templates. They are excluded from version control via .gitignore.

Path	Description
carrental/	Repository root
carrental/backend/	Node.js / Express API server
carrental/frontend/	React.js single-page application
carrental/README.md	Project setup and usage guide
carrental/package.json	Root scripts (install-all, dev, seed)
carrental/.gitignore	Version control exclusions

4.2 Backend Directory Structure

Directory / File	Responsibility
backend/server.js	Application entry point; mounts routes, middleware, error handlers
backend/config/db.js	MongoDB connection with retry logic and connection logging
backend/models/	Mongoose schema definitions (User, Car, Booking, Review, Payment)
backend/controllers/	Business logic handlers invoked by route definitions
backend/routes/	Express route declarations; maps HTTP verbs + paths to controllers
backend/middleware/	Authentication guard (protect) and role check (admin) middleware
backend/utils/	Shared utilities: token generation, date calculations, price logic
backend/seed/seeder.js	Database seeder; populates demo cars, users, bookings, reviews

backend/.env.example	Template for required environment variable configuration

4.3 Frontend Directory Structure

Directory / File	Responsibility
frontend/src/App.js	Root component; configures React Router and top-level layout
frontend/src/index.js	Application bootstrap; renders root component, wraps with providers
frontend/src/index.css	Global CSS custom properties (design tokens) and utility classes
frontend/src/context/AuthContext.js	Global authentication state; user session, wishlist toggle
frontend/src/services/api.js	Axios instance with interceptors; all API call functions
frontend/src/utils/helpers.js	Date formatting, currency formatting, status color mapping
frontend/src/components/common/	Navbar, Footer, Loader, StarRating, ProtectedRoute, AdminRoute
frontend/src/components/cars/	CarCard grid item, CarFilter sidebar
frontend/src/components/admin/	AdminLayout with collapsible sidebar navigation
frontend/src/pages/	All page-level route components (15 pages total)
frontend/src/pages/admin/	Admin-specific pages: Dashboard, Cars, Users, Bookings

5. Data Models

DriveElite employs five primary Mongoose schemas. All schemas include createdAt and updatedAt timestamp fields automatically via the timestamps: true option. Relationships between documents use ObjectId references (ref) and are resolved via Mongoose's .populate() method.

5.1 User Schema

Field	Type	Required	Notes
name	String	Yes	Trimmed; validated non-empty
email	String	Yes	Unique, lowercase, trimmed; used as login identifier
password	String (hashed)	Yes	Min 6 chars; auto-hashed via pre-save bcrypt hook
phone	String	No	Optional contact number
avatar	String	No	URL; defaults to ui-avatars.com generated avatar
role	Enum	Yes	'user' | 'admin'; defaults to 'user'
wishlist	ObjectId[]	No	Array of Car references
address	Object	No	street, city, state, country sub-fields
drivingLicense	String	No	License number for record-keeping
isActive	Boolean	Yes	Defaults true; admin can deactivate accounts

5.2 Car Schema

Field	Type	Required	Notes
name	String	Yes	Display name e.g. 'Tesla Model S'
brand	String	Yes	Manufacturer e.g. 'Tesla'
model	String	Yes	Model identifier e.g. 'Model S'
year	Number	Yes	Manufacturing year
category	Enum	Yes	Economy | Premium | Luxury | SUV | Hatchback | Van
type	Enum	Yes	Sedan | SUV | Hatchback | Luxury | Van | Convertible
pricePerDay	Number	Yes	Daily rental rate in USD
images	String[]	No	Array of image URLs; first image used as thumbnail
features	String[]	No	e.g. ['Autopilot', 'Premium Sound']
specs	Object	Yes	fuelType, transmission, seats, mileage, engine, horsepower

location	String	Yes	City/region where vehicle is based
availability	Boolean	Yes	Defaults true; set false when removed from listing
bookedDates	Object[]	No	Array of {startDate, endDate, bookingId} for conflict check
rating	Number	No	Auto-calculated average from Reviews; defaults 0
numReviews	Number	No	Auto-counted; updated by Review static method

5.3 Booking Schema

Field	Type	Required	Notes
user	ObjectId (User)	Yes	Reference to booking owner
car	ObjectId (Car)	Yes	Reference to rented vehicle
pickupDate	Date	Yes	Rental start date
dropoffDate	Date	Yes	Rental end date
pickupLocation	String	Yes	Customer-selected pickup point
dropoffLocation	String	No	Defaults to pickupLocation if not specified
totalDays	Number	Yes	Computed: diff(dropoffDate, pickupDate)
pricePerDay	Number	Yes	Snapshot of rate at booking time
totalAmount	Number	Yes	pricePerDay × totalDays
status	Enum	Yes	pending | confirmed | active | completed | cancelled
paymentStatus	Enum	Yes	pending | paid | refunded | failed
bookingNumber	String	Yes	Unique; auto-generated 'BK' + timestamp + random suffix
cancellationReason	String	No	Populated when booking is cancelled

5.4 Review Schema

Reviews are constrained to one-per-user-per-vehicle via a compound unique index on { user, car }. Average ratings are recalculated automatically via a post-save static method that aggregates all reviews for the affected Car document.

Field	Type	Required	Notes
user	ObjectId (User)	Yes	Reviewer reference
car	ObjectId (Car)	Yes	Reviewed vehicle reference
booking	ObjectId (Booking)	No	Optional booking reference for verified review
rating	Number	Yes	1–5 integer scale
title	String	Yes	Short review headline
comment	String	Yes	Full review body text

6. API Reference

The DriveElite API is a RESTful HTTP API served at the base URL configured via the REACT_APP_API_URL environment variable (default: http://localhost:5000/api). All endpoints return JSON with the following response envelope:

> **i** All protected endpoints require the Authorization: Bearer <token> HTTP header. Absent or invalid tokens return HTTP 401 Unauthorized.

6.1 Authentication Endpoints

Method	Path	Auth	Description
POST	/api/auth/register	Public	Create new user account; returns JWT
POST	/api/auth/login	Public	Authenticate with email + password; returns JWT
GET	/api/auth/me	User	Retrieve authenticated user profile with wishlist

6.2 Vehicle Endpoints

Method	Path	Auth	Description
GET	/api/cars	Public	List vehicles; supports query filters (location, category, pricePerDay range, fuelType, transmission, search, available, page, limit)
GET	/api/cars/featured	Public	Top 6 vehicles by rating, availability=true
GET	/api/cars/:id	Public	Vehicle detail by MongoDB ObjectId
POST	/api/cars	Admin	Create new vehicle listing
PUT	/api/cars/:id	Admin	Update vehicle fields; partial update supported
DELETE	/api/cars/:id	Admin	Permanently remove vehicle listing

6.3 Booking Endpoints

Method	Path	Auth	Description
POST	/api/bookings	User	Create booking; validates date conflicts, calculates totals, reserves dates
GET	/api/bookings/my	User	All bookings for authenticated user, newest first
GET	/api/bookings/all	Admin	All platform bookings with user and car data
GET	/api/bookings/:id	User	Single booking detail; user can only view own bookings
PUT	/api/bookings/:id/cancel	User	Cancel pending or confirmed booking; removes reserved dates from car
PUT	/api/bookings/:id/status	Admin	Update booking lifecycle status

6.4 Payment Endpoints

Method	Path	Auth	Description
POST	/api/payments/create-intent	User	Create Stripe PaymentIntent; returns clientSecret
POST	/api/payments/confirm	User	Confirm payment; updates Booking and Payment records
POST	/api/payments/mock	User	Bypass Stripe; sets paymentStatus=paid (test/demo use only)
GET	/api/payments/my	User	Payment history for authenticated user

6.5 Review Endpoints

Method	Path	Auth	Description
POST	/api/reviews	User	Submit review; enforces one-per-user-per-car constraint
GET	/api/reviews/car/:carId	Public	All approved reviews for a vehicle, newest first
DELETE	/api/reviews/:id	User/Admin	Delete own review; admin can delete any review

6.6 Admin Endpoints

Method	Path	Auth	Description
GET	/api/admin/stats	Admin	Dashboard aggregates: counts, revenue, monthly trend, category breakdown
GET	/api/admin/users	Admin	All registered users sorted by registration date
DELETE	/api/admin/users/:id	Admin	Remove non-admin user account
PUT	/api/admin/users/:id/toggle	Admin	Toggle user isActive flag

7. Security Architecture

Security is a foundational design concern in DriveElite. The following controls are implemented across the authentication, authorization, data, and transport layers.

7.1 Authentication

User authentication is implemented using JSON Web Tokens (JWT) as defined by RFC 7519. Upon successful login or registration, the server issues a signed token containing the user's MongoDB ObjectId. The token is signed with a secret key stored in the JWT_SECRET environment variable and expires after 30 days (configurable via JWT_EXPIRE).

- Tokens are transmitted in the Authorization: Bearer <token> header on all authenticated requests.
- The frontend stores the token in localStorage and attaches it to all Axios requests via a default header.
- An Axios response interceptor on the frontend automatically redirects users to /login and clears stored tokens upon receiving a 401 response.

7.2 Password Security

User passwords are never stored in plaintext. The bcryptjs library applies the bcrypt algorithm with a cost factor of 12 rounds during the User model's pre-save hook. This means passwords are hashed before they reach the database layer, even in the event of a developer error or direct database write.

> **i** The cost factor of 12 is deliberately chosen to balance security and performance. At this factor, hashing takes approximately 250–400ms per operation, making brute-force attacks computationally expensive.

7.3 Authorization & Role-Based Access Control

DriveElite implements two authorization levels:

- User: Can manage own profile, wishlist, bookings, and reviews. Cannot access admin endpoints.
- Admin: Has unrestricted access to all platform data including vehicle management, user governance, and booking status updates.

Authorization is enforced at the route level via two Express middleware functions:

- protect: Verifies the JWT, loads the associated user from MongoDB, and attaches it to req.user. Rejects requests with missing, expired, or invalid tokens with HTTP 401.
- admin: Checks req.user.role === 'admin'. Rejects non-admin users with HTTP 403 Forbidden.

7.4 Data Validation

Input validation occurs at multiple layers:

- Mongoose schema validation enforces type correctness, required fields, enum constraints, and uniqueness on every write operation.
- Controller-level validation checks for required fields before attempting database operations, returning descriptive 400 Bad Request responses on failure.
- Frontend form validation provides immediate user feedback before API calls are made, reducing unnecessary server load.

7.5 CORS Configuration

Cross-Origin Resource Sharing (CORS) is configured on the Express server to accept requests only from the origin specified in the CLIENT_URL environment variable. This prevents unauthorized browser-based cross-origin requests to the API. Credentials are enabled to support cookie-based flows in future iterations.

7.6 Security Recommendations for Production

> ⚠ The following controls are recommended before deploying to a production environment. They are not enabled in the development build.

- Enforce HTTPS: Use TLS certificates (Let's Encrypt) on all API endpoints. Redirect HTTP to HTTPS.
- Rate Limiting: Apply express-rate-limit on authentication endpoints (login, register) to prevent credential stuffing attacks.
- Helmet.js: Add the helmet middleware to set secure HTTP response headers (CSP, X-Content-Type-Options, HSTS, etc.).
- Input Sanitization: Add express-mongo-sanitize to prevent NoSQL injection via malicious query operators.
- Audit Logging: Implement structured logging (Winston + CloudWatch/Datadog) for all authentication events and admin actions.
- Secrets Rotation: Rotate JWT_SECRET and STRIPE_SECRET_KEY on a scheduled basis and upon suspected compromise.

8. Feature Specification

8.1 User Registration & Authentication

New users register via a split-panel form requiring full name, email address, password (minimum 6 characters), and an optional phone number. Passwords are confirmed client-side before submission. Upon successful registration, the API returns a JWT token and user object; the frontend stores the token and updates global auth state, redirecting the user to the home page.

Returning users authenticate via the login form. The API validates credentials against the hashed password in MongoDB. Upon success, the same JWT token pattern is applied. Incorrect credentials return a generic 'Invalid email or password' error to prevent user enumeration attacks.

8.2 Vehicle Discovery & Filtering

The vehicle browsing page provides a sidebar filter panel with the following independent filter dimensions:

- Full-text search across vehicle name, brand, and model fields (MongoDB regex query)
- Location: partial string match against the car's location field
- Category: Economy, Premium, Luxury, SUV, Hatchback, Van
- Body Type: Sedan, SUV, Hatchback, Luxury, Van, Convertible
- Fuel Type: Petrol, Diesel, Electric, Hybrid
- Transmission: Automatic, Manual
- Price Range: minimum and maximum daily rate
- Availability toggle: show only currently available vehicles

Results are paginated at 9 vehicles per page with client-side page navigation. Sort options include default (newest first), price ascending, and price descending. Results are displayed in a responsive grid or list layout selectable by the user.

8.3 Booking Flow

The booking flow is a three-step process: vehicle selection → booking details → payment. The process is designed to minimize friction while ensuring data integrity.

1. User selects a vehicle and clicks 'Book Now' on the vehicle detail page.
2. User completes the booking form: pickup date, drop-off date, pickup location, optional drop-off location, and notes.
3. The API validates the submitted date range against existing bookedDates on the vehicle using an overlap detection algorithm.
4. If no conflict exists, a Booking document is created with status: 'pending'. The vehicle's bookedDates array is updated atomically to reserve the dates.
5. The user is redirected to the payment page for the newly created booking.

Date conflict detection uses the following logic: a conflict exists when the requested pickup date is before an existing end date AND the requested drop-off date is after an existing start date. This correctly handles all overlap scenarios including partial overlaps, complete containment, and exact matches.

8.4 Payment Processing

DriveElite integrates with the Stripe Payments API for card processing. The payment page collects card details and submits them to Stripe's client-side SDK. The server creates a PaymentIntent via the Stripe server-side SDK and returns the clientSecret to the frontend for confirmation.

A mock payment gateway is provided for demonstration and testing environments where Stripe credentials are not configured. The mock gateway bypasses Stripe entirely and directly marks the booking as paid. The following Stripe test card numbers are supported:

Card Number	Outcome	Use Case
4242 4242 4242 4242	**Success**	Standard successful payment
4000 0000 0000 0002	**Decline**	Card declined simulation

8.5 Review & Rating System

Authenticated users may submit one review per vehicle. Reviews consist of a star rating (1–5), a title, and a free-text comment. Upon submission, a post-save database hook recalculates the vehicle's aggregate average rating and review count, keeping Car documents consistently current without requiring expensive query-time aggregation.

Users may delete their own reviews. Administrators may delete any review. Deletion triggers the same rating recalculation hook to maintain data integrity.

8.6 Wishlist Management

The wishlist is stored as an array of Car ObjectIds on the User document. Toggle operations (add/remove) are performed via a single PUT endpoint that checks whether the vehicle ID is already present in the array and either adds or removes it accordingly. The updated wishlist is returned in the response and immediately reflected in global auth state without requiring a page refresh.

9. Administration Console

The administration console is accessible to users with role: 'admin' only. Access is enforced at both the frontend route level (AdminRoute component redirects non-admins) and the API level (admin middleware on all /api/admin/* and relevant routes).

9.1 Analytics Dashboard

The admin dashboard presents an at-a-glance operational overview using eight KPI cards and two data visualizations.

- KPI cards: Total Cars, Available Cars, Total Users, Total Bookings, Confirmed Bookings, Completed Bookings, Cancelled Bookings, Total Revenue
- Monthly Revenue Bar Chart: 12-month revenue trend using MongoDB aggregation pipeline grouped by calendar month
- Bookings by Category Donut Chart: Distribution of bookings across vehicle categories using a join aggregation between Bookings and Cars collections
- Recent Bookings Table: Last 5 bookings with customer name, vehicle, amount, status, and date

9.2 Fleet Management

Administrators can perform full CRUD operations on vehicle listings through a modal-based form. The form is organized into four logical sections:

- Basic Information: name, brand, model, year, registration number, color, location, price, category, and body type
- Specifications: fuel type, transmission, seats, mileage, engine displacement, horsepower, 0-60 time, and top speed
- Images: up to three image URLs with inline preview on URL entry
- Features & Description: comma-delimited feature tags and free-text vehicle description

Vehicle availability can be toggled independently of active bookings, allowing administrators to temporarily remove vehicles from the booking pool for maintenance without cancelling existing reservations.

9.3 User Management

The user management interface lists all registered accounts with filterable views by status (all, active, inactive) and role (user, admin). Administrators can:

- Toggle user account active status (deactivation prevents login without deleting data)
- Permanently delete non-admin user accounts

> ⚠ Administrator accounts are protected from deletion. Attempting to delete a user with role: 'admin' returns HTTP 400 Bad Request.

9.4 Booking Management

Administrators can view all platform bookings with comprehensive filtering by status. For each booking, a detail modal displays full booking information including customer contact details, vehicle information, pickup/drop-off dates and locations, payment status, and any customer notes. Booking lifecycle status can be updated directly from the table view via a select control.

10. Installation & Configuration

10.1 Prerequisites

Ensure the following software is installed on the development machine before proceeding:

Prerequisite	Minimum Version	Purpose
Node.js	18.0 LTS	JavaScript runtime for backend and frontend tooling
npm	9.0	Package manager (bundled with Node.js)

MongoDB	6.0 (local) or Atlas	Database (local install or cloud Atlas cluster)
Git	2.x	Version control

10.2 Environment Configuration

Two .env files must be created before starting the application — one for the backend and one for the frontend. Copy the provided .env.example files as starting templates.

Backend Environment Variables (backend/.env)

Variable	Required	Description
PORT	No	HTTP server port; defaults to 5000
MONGODB_URI	Yes	Full MongoDB connection string including credentials
JWT_SECRET	Yes	Cryptographically random string; minimum 32 characters recommended
JWT_EXPIRE	No	Token expiry duration; defaults to '30d'
STRIPE_SECRET_KEY	No*	Stripe server-side secret key (sk_test_…); mock mode used if absent
CLIENT_URL	Yes	Frontend origin URL for CORS; e.g. http://localhost:3000
NODE_ENV	No	'development' enables Morgan logging; 'production' suppresses stack traces

Frontend Environment Variables (frontend/.env)

Variable	Required	Description
REACT_APP_API_URL	Yes	Backend API base URL; e.g. http://localhost:5000/api
REACT_APP_STRIPE_PUBLISHABLE_KEY	No*	Stripe client-side publishable key (pk_test_…); mock mode if absent

10.3 Installation Steps

6. Clone the repository: git clone <repository-url> && cd carrental
7. Install backend dependencies: cd backend && npm install
8. Install frontend dependencies: cd ../frontend && npm install

9. Create backend environment file: cp .env.example .env and populate all required variables
10. Create frontend environment file: cp .env.example .env and populate REACT_APP_API_URL
11. Seed the database with demo data: cd backend && npm run seed
12. Start the backend server: npm run dev (uses nodemon for hot reload)
13. In a new terminal, start the frontend: cd frontend && npm start
14. Open http://localhost:3000 in a browser

10.4 Demo Credentials

The seeder script creates the following accounts for immediate use:

Role	Email	Password	Access Level
Admin	admin@carrental.com	admin123	Full platform access including admin console
Standard User	john@example.com	user123	Vehicle browsing, booking, wishlist, reviews
Standard User	sarah@example.com	user123	Vehicle browsing, booking, wishlist, reviews
Standard User	mike@example.com	user123	Vehicle browsing, booking, wishlist, reviews

> ⚠ Change all default credentials immediately upon deploying to any internet-accessible environment.

11. Error Handling & Logging

11.1 Backend Error Architecture

The Express application uses a centralized error handling strategy. All asynchronous controller functions are wrapped by the express-async-handler utility, which catches rejected promises and forwards them to Express's built-in error pipeline, eliminating repetitive try/catch boilerplate.

Two global middleware functions are registered after all route definitions in server.js:

- notFound: Catches requests that do not match any defined route and creates a 404 error with the requested path.
- errorHandler: Formats all errors into a consistent JSON response envelope containing a success flag (false), message string, and optionally a stack trace (suppressed in production).

11.2 Standard Error Response Format

All API errors follow a consistent JSON response structure regardless of the error source:

```
{   "success": false,   "message": "Human-readable error description",   "stack":
"...stacktrace (development only)..."}
```

11.3 HTTP Status Code Conventions

Status Code	Meaning	Example Scenarios
200 OK	Successful retrieval or update	GET /api/cars, PUT /api/bookings/:id/status
201 Created	Resource successfully created	POST /api/auth/register, POST /api/bookings
400 Bad Request	Validation failure or business rule violation	Missing required fields, duplicate booking dates
401 Unauthorized	Missing or invalid authentication token	Accessing protected routes without JWT
403 Forbidden	Authenticated but insufficient privileges	Non-admin accessing /api/admin/* endpoints
404 Not Found	Requested resource does not exist	Invalid car ID, booking ID
500 Internal Server Error	Unexpected server-side error	Database connectivity failure

11.4 Frontend Error Handling

The Axios instance configured in src/services/api.js includes a response interceptor that:

- Detects HTTP 401 responses and clears the stored JWT token from localStorage.
- Redirects the browser to /login if the current path is not already /login.
- Passes all other errors through the Promise rejection chain for component-level handling.

Individual components and page-level handlers catch API errors and display user-facing notifications via the React Hot Toast notification system. Error messages are extracted from the err.response.data.message field when present, falling back to a generic descriptive message.

12. Quality Assurance & Testing

The following testing strategy is recommended for the DriveElite platform. The seed data and demo accounts provided in the seeder script serve as a baseline dataset for all manual and automated testing activities.

12.1 Manual Testing Checklist — User Flows

Test Case	Steps	Expected Result
User Registration	Navigate to /register, complete form, submit	Redirected to home; user logged in
Login — Valid Credentials	Navigate to /login, enter john@example.com / user123	Redirected to home; navbar shows user name
Login — Invalid Password	Enter correct email, wrong password	Toast error: 'Invalid email or password'
Car Search by Location	Navigate to /cars, enter 'New York' in location filter	Only New York vehicles displayed
Date Conflict Booking	Attempt to book car for dates already reserved	Error: 'Car is not available for selected dates'
Successful Booking	Book available car with future dates	Redirected to payment page; booking in DB
Mock Payment — Success	On payment page, use card 4242 4242 4242 4242	Redirected to /payment/success; booking confirmed
Mock Payment — Failure	On payment page, use card 4000 0000 0000 0002	Redirected to /payment/failure
Cancel Booking	In dashboard, cancel a confirmed booking	Booking status changes to cancelled; dates freed
Wishlist Toggle	Click heart on car card; check wishlist page	Car appears/disappears from wishlist
Admin — Add Car	Login as admin, navigate to /admin/cars, add new car	Car appears in listing immediately
Admin — Delete User	Navigate to /admin/users, delete a non-admin user	User removed from list; login fails
Protected Route	Access /dashboard without login	Redirected to /login with return URL
Admin Route	Access /admin as regular user	Redirected to /

12.2 Recommended Automated Testing

The following automated testing layers are recommended for production readiness:

- Unit Tests: Jest + React Testing Library for isolated component rendering, form validation logic, and utility functions.
- API Integration Tests: Supertest with a dedicated test MongoDB database (e.g., mongodb-memory-server) for endpoint contract testing.
- End-to-End Tests: Cypress or Playwright for full user flow simulation including auth, booking, and payment flows in a browser environment.

13. Performance Considerations

13.1 Database Indexing

The following MongoDB indexes are recommended to ensure query performance at scale. Without appropriate indexes, queries on large collections will perform full collection scans, resulting in O(n) latency growth as data volume increases.

Collection	Index Fields	Type	Rationale
users	email	Unique	Login lookup; enforces uniqueness
cars	location	Single	Location filter on listing page
cars	category, type	Compound	Category + type filter combination
cars	pricePerDay	Single	Price range filter and sort
cars	availability, rating	Compound	Featured cars query
bookings	user	Single	User's booking history
bookings	car, status	Compound	Car availability check and status filter
bookings	bookingNumber	Unique	Booking lookup by number
reviews	user, car	Unique Compound	One-review-per-user-per-car constraint
reviews	car	Single	Fetch all reviews for a vehicle

13.2 Frontend Optimization

- Code Splitting: React Router's lazy() and Suspense can be applied to page components to reduce the initial bundle size and improve time-to-interactive.
- Image Optimization: Vehicle images are served from external CDN URLs. For production, images should be processed through an image optimization service (e.g., Cloudinary, imgix) to serve appropriately sized and formatted assets.
- Pagination: The API enforces a default page size of 9 vehicles, preventing large data payloads from degrading browser rendering performance.
- Memoization: useCallback is applied to the fetchCars function in CarsPage to prevent unnecessary re-fetches when unrelated state changes occur.

13.3 API Response Optimization

- Selective population: Mongoose .populate() calls specify field projections (e.g., 'name brand images pricePerDay') to return only necessary fields from related documents, reducing payload sizes.
- Aggregation pipelines: Dashboard analytics use MongoDB aggregation rather than fetching all documents and computing in JavaScript, keeping CPU-intensive work close to the data.

14. Deployment Guide

14.1 Production Build

Before deploying to production, generate an optimized React production build:

15. Navigate to the frontend directory: cd frontend
16. Run the build command: npm run build
17. The build/ directory contains the optimized static bundle ready for CDN deployment.
18. Set NODE_ENV=production in the backend environment before starting the server to disable development-only features including Morgan logging and stack trace exposure.

14.2 Recommended Deployment Topology

Service	Recommended Provider	Alternative
Frontend	Vercel (automatic CI/CD from Git)	Netlify, AWS S3 + CloudFront
Backend API	Railway or Render (container-based)	AWS EC2, Heroku, DigitalOcean App Platform
Database	MongoDB Atlas (M10+ for production)	Self-hosted replica set on AWS
Payment	Stripe (live mode keys)	PayPal, Razorpay for INR markets

CDN / Assets	Cloudinary for vehicle images	AWS S3 + CloudFront

14.3 Environment Promotion Strategy

A three-environment promotion strategy is recommended:

- Development: Local developer machines running against a local or shared Atlas development cluster. All environment variables point to non-production services.
- Staging: Cloud-hosted environment mirroring production topology. Used for integration testing, QA sign-off, and UAT. Stripe test mode keys used.
- Production: Live environment serving real users. Stripe live mode keys. MongoDB Atlas production cluster with automated daily backups retained for 7 days minimum.

14.4 CI/CD Pipeline (Recommended)

- Source Control: Feature branches merged to main via pull request with mandatory code review.
- Automated Testing: Jest unit tests and Supertest API tests run on every pull request via GitHub Actions.
- Build Validation: React production build verified on every merge to main.
- Staging Deployment: Automatic deployment to staging environment on merge to main.
- Production Deployment: Manual promotion gate from staging to production after QA approval.

15. Troubleshooting Guide

15.1 Common Issues & Resolutions

Symptom	Likely Cause	Resolution
'MongooseServerSelectionError' on startup	MongoDB not running or incorrect URI	Verify MongoDB service is running; check MONGODB_URI in .env
API returns 401 on all requests	JWT_SECRET mismatch between token issue and verification	Ensure JWT_SECRET is identical in both .env and the running process
Frontend cannot connect to API	REACT_APP_API_URL misconfigured	Confirm the value matches the backend URL and port; restart dev server after .env changes
Stripe payment fails in test mode	Wrong test card or missing publishable key	Use 4242 4242 4242 4242; ensure

		REACT_APP_STRIPE_PUBLISHABLE_KEY is set
Seeder fails with 'duplicate key' error	Database already seeded	Drop existing collections via MongoDB Compass or run db.dropDatabase() before re-seeding
Admin panel inaccessible after login	User role is 'user', not 'admin'	Log in with admin@carrental.com or update role via MongoDB directly
CORS error in browser console	CLIENT_URL does not match frontend origin	Update CLIENT_URL in backend .env to exact frontend URL including port
Images not loading	Unsplash URL throttling or network restriction	Replace image URLs with locally hosted alternatives or a Cloudinary account

15.2 Debugging Tips

- Backend: Run NODE_ENV=development npm run dev to enable Morgan request logging and full stack traces in error responses.
- Frontend: Open Browser DevTools → Network tab to inspect API requests and responses. Console errors from the Axios interceptor will identify 401/403 patterns.
- Database: Use MongoDB Compass or the Atlas GUI to inspect documents directly. The Mongoose debug mode can be enabled via mongoose.set('debug', true) in config/db.js.
- Payments: Use the Stripe Dashboard test mode event log to inspect PaymentIntent status transitions.

Appendix A: Glossary

Term	Definition
API	Application Programming Interface — a defined contract for how software components communicate
CORS	Cross-Origin Resource Sharing — a browser security mechanism controlling cross-origin HTTP requests
CRUD	Create, Read, Update, Delete — the four fundamental database operations
DXA	Document eXact Alignment unit — 1/1440 inch; used in Word/OOXML documents for measurements
JWT	JSON Web Token — a compact, URL-safe means of representing authentication claims

MERN	MongoDB, Express.js, React.js, Node.js — a full-stack JavaScript technology combination
ODM	Object Document Mapper — maps database documents to application objects (Mongoose for MongoDB)
ORM	Object Relational Mapper — analogous to ODM but for relational databases
REST	Representational State Transfer — an architectural style for HTTP APIs using standard verbs
SPA	Single Page Application — a web app loading a single HTML page and updating content dynamically
SSR	Server-Side Rendering — rendering HTML on the server before sending to the client
TLS	Transport Layer Security — cryptographic protocol providing secure communication over networks

Appendix B: Seed Data Summary

The database seeder (npm run seed) populates the following records upon execution:

Collection	Count	Details
Users	4	1 admin + 3 standard users with realistic profiles
Cars	12	Mix of Economy, Premium, Luxury, SUV and Hatchback vehicles with real specs
Bookings	3	1 completed (with payment), 1 confirmed (with payment), 1 upcoming
Payments	3	All mock payments; status 'succeeded'
Reviews	3	One per completed booking; triggers rating recalculation

Appendix C: Third-Party Service Dependencies

Service	Purpose	Required ?	Configuration
Stripe	Payment processing	No (mock fallback)	STRIPE_SECRET_KEY + REACT_APP_STRIPE_PUBLISHABLE_KEY

MongoDB Atlas	Cloud database	No (local MongoDB works)	MONGODB_URI with Atlas connection string
UI Avatars	Default user avatars	No	Public service; no API key required
Unsplash	Vehicle listing images	No	Public URLs in seed data; no key required
Google Fonts	Plus Jakarta Sans + DM Serif Display	No	Loaded via HTML <link>; falls back to system fonts

Appendix D: Contact & Support

For questions regarding this documentation or the DriveElite platform, contact the Engineering team through the following channels:

Channel	Contact	Response SLA
Engineering Email	engineering@driveelite.com	2 business days
Security Issues	security@driveelite.com	4 hours (critical) / 24 hours (non-critical)
Documentation Feedback	docs@driveelite.com	5 business days